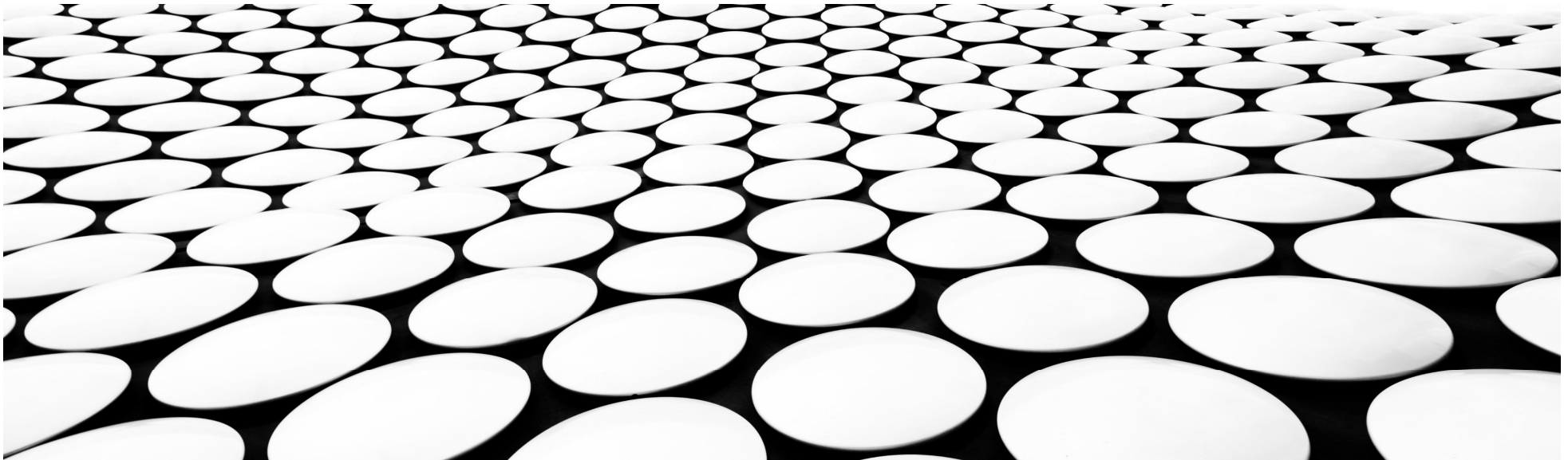
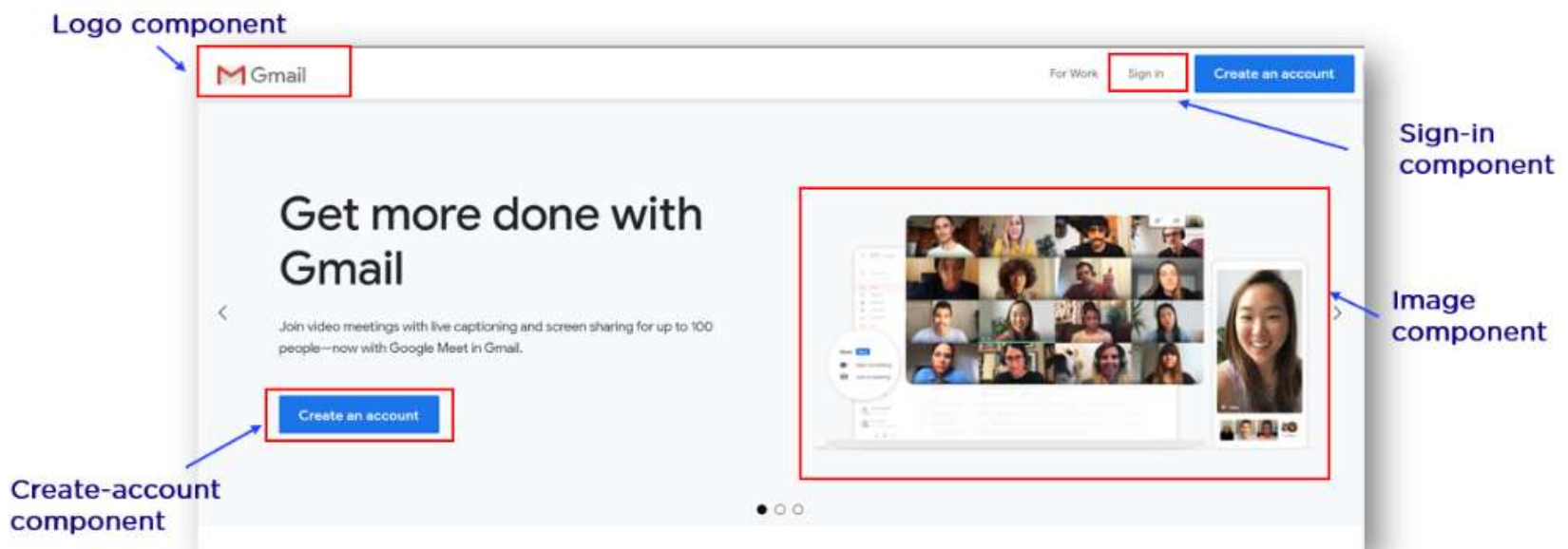

ANGULAR SERVICES

NGUYEN THI LOAN



WHAT IS THE NEED FOR ANGULAR SERVICES?

- ❑ The user interface of the application is developed by embedding several components into the main component.

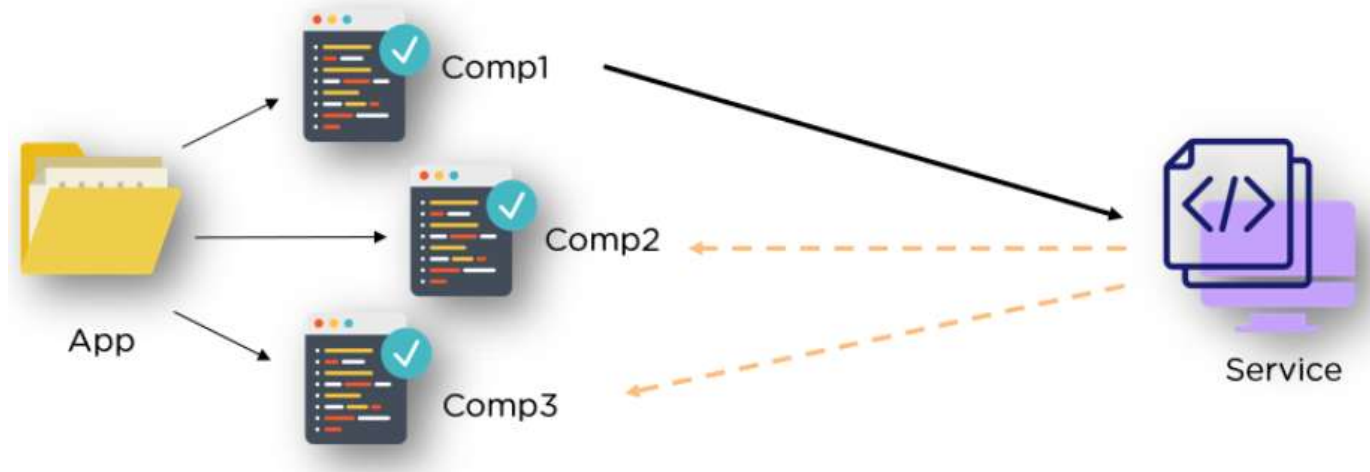


WHAT IS THE NEED FOR ANGULAR SERVICES?

- ❑ Services avoid rewriting of code. A service can be written once and injected into all the components that use that service
- ❑ A service could be a function, variable, or feature that an application needs

WHAT ARE ANGULAR SERVICES?

- ❑ Angular services are objects that get instantiated just once during the lifetime of an application.
- ❑ They contain methods that maintain data throughout the life of an application, i.e., data is available all the time.



FEATURES OF ANGULAR SERVICES

- ❑ Services in Angular are simply typescript classes with the **@injectable** decorator. This decorator tells angular that the class is a service and can be injected into components that need that service
- ❑ Services are used to share a single piece of code across multiple components
- ❑ Services are used to interact with the backend

ANGULAR SERVICES

- ❑ You would create and use Angular services when:
 - You need to retrieve data from or send data to your server.
 - You need to encapsulate application logic that is not specific to any one component, or logic that can be reused across components
 - You need to share data across components, especially across components that may or may not know about each other.

CREATING AN ANGULAR SERVICE

- ❑ Extend the stock market application using Angular services. We will accomplish the following things in our stock market application:
 - Fetch the list of stocks to show from a service, instead of hardcoding it in the component.
 - When we create a stock, we will send it to the service.
 - When we create a stock, we want it to show up in our list of stocks.

THE STOCK MARKET APPLICATION

- ❑ We will add a **StockListComponent** that fetches a list of stocks, and displays them using the **StockItemComponent**.
- ❑ We will build the **StockService** that act as a layer behind both the components.

DIGGING INTO THE EXAMPLE

- First, we will change the template for the *StockItemComponent* slightly so that displays both an Add to Favorite and a Remove from Favorite button depending on the state of the stock.

```
<div class="stock-container">
  <div class="name">{{stock.name + ' (' + stock.code + ')'}}</div>
  <div class="exchange">{{stock.exchange}}</div>
  <div class="price"
    [class.positive]="stock.isPositiveChange()"
    [class.negative]="!stock.isPositiveChange()">$ {{stock.price}}</div>
  <button (click)="onToggleFavorite($event)"
    *ngIf="!stock.favorite">Add to Favorite</button>
  <button (click)="onToggleFavorite($event)"
    *ngIf="stock.favorite">Remove from Favorite</button>
</div>
```

THE STOCK MARKET APPLICATION

- Next, create the *StockListComponent* before we try creating and integrating our *StockService*:
 - ng g component stock/stock-list

```
export class StockListComponent implements OnInit {  
  
    public stocks: Stock[];  
    constructor() { }  
  
    ngOnInit() {  
        this.stocks = [  
            new Stock('Test Stock Company', 'TSC', 85, 80, 'NASDAQ'),  
            new Stock('Second Stock Company', 'SSC', 10, 20, 'NSE'),  
            new Stock('Last Stock Company', 'LSC', 876, 765, 'NYSE')  
        ];  
    }  
  
    onToggleFavorite(stock: Stock) {  
        console.log('Favorite for stock ', stock, ' was triggered');  
        stock.favorite = !stock.favorite;  
    }  
}
```

THE STOCK MARKET APPLICATION

- Let's now look at its corresponding template, in *stocklist.component.html*:

```
<app-stock-item *ngFor="let stock of stocks" [stock]="stock"  
                (toggleFavorite)="onToggleFavorite($event)">  
</app-stock-item>
```

THE STOCK MARKET APPLICATION

- The component class *app.component.ts* can be modified to:

```
import { Component, OnInit } from '@angular/core';
```

```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent implements OnInit {  
  title = 'Stock Market App';  
  
  ngOnInit(): void {  
  }  
}
```

app.component.html

```
<h1>  
  {{title}}  
</h1>  
<app-stock-list></app-stock-list>  
<app-create-stock></app-create-stock>
```

CREATE SERVICE

□ We can create the service using the Angular CLI as follows:

- `ng g service services/stock`
- This will generate two files, a skeleton *stock-service.ts* and a dummy test for it in *stockservice.spec.ts*.

```
import { Injectable } from '@angular/core';

@Injectable()
export class StockService {

    constructor() { }

}
```

CREATE SERVICE

- Let's define our StockService to continue returning mock data.
- We will edit the *stock.service.ts* file as follows:

```
import { Injectable } from '@angular/core';
import { Stock } from 'app/model/stock';

@Injectable()
export class StockService {

  private stocks: Stock[];
  constructor() {
    this.stocks = [
      new Stock('Test Stock Company', 'TSC', 85, 80, 'NASDAQ'),
      new Stock('Second Stock Company', 'SSC', 10, 20, 'NSE'),
      new Stock('Last Stock Company', 'LSC', 876, 765, 'NYSE')
    ];
  }
  getStocks() : Stock[] {
    return this.stocks;
  }
  createStock(stock: Stock) {
    let foundStock = this.stocks.find(each => each.code === stock.code);
    if (foundStock) {
      return false;
    }
    this.stocks.push(stock);
    return true;
  }
  toggleFavorite(stock: Stock) {
    let foundStock = this.stocks.find(each => each.code === stock.code);
    foundStock.favorite = !foundStock.favorite;
  }
}
```

REGISTERING SERVICE

- Let's edit the *app.module.ts* file to define the provider as follows

```
import { StockService } from 'app/services/stock.service';

@NgModule({
  declarations: [
    AppComponent,
    StockItemComponent,
    CreateStockComponent,
    StockListComponent
  ],
  imports: [
    BrowserModule,
    FormsModule,
    HttpClientModule
  ],
  providers: [
    StockService
  ],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

- 1 Registering the provider for StockService

USING SERVICE

- We are ready to start using the service, so first we will use it in the newly created *StockListComponent*.
- Let's change the *stocklist.component.ts* file as follows:

```
import { Component, OnInit } from '@angular/core';
import { StockService } from 'app/services/stock.service';
import { Stock } from 'app/model/stock';

@Component({
  selector: 'app-stock-list',
  templateUrl: './stock-list.component.html',
  styleUrls: ['./stock-list.component.css']
})
export class StockListComponent implements OnInit {

  public stocks: Stock[];
  constructor(private stockService: StockService) { } ❶

  ngOnInit() {
    this.stocks = this.stockService.getStocks(); ❷
  }

  onToggleFavorite(stock: Stock) {
    console.log('Favorite for stock ', stock, ' was triggered');
    this.stockService.toggleFavorite(stock); ❸
  }
}
```

- ❶ Inject the StockService into the component
- ❷ Use the StockService to get the list of stocks
- ❸ Use the StockService to toggle the favorite status on a stock

THE RESULT



THE EXAMPLE

- Let's continue with the minor changes to the *CreateStockComponent* to finish the service integration.
- We will edit *createstock.component.html* as follows:

```
<h2>Create Stock Form</h2>
```

```
<div *ngIf="message">{{message}}</div>
```

```
<div class="form-group">
```

```
  <!-- Rest of the form omitted for brevity -->
```

```
  <!-- No change from the base code -->
```

```
</div>
```

```
<h4>Stock Name is {{stock | json}}</h4>
```

- 1 Display a message if it exists

THE EXAMPLE

- Now, we can change the component in *createstock.component.ts* to integrate with the StockService:

```
import { Component, OnInit } from '@angular/core';
import { Stock } from 'app/model/stock';
import { StockService } from 'app/services/stock.service';

@Component({
  selector: 'app-create-stock',
  templateUrl: './create-stock.component.html',
  styleUrls: ['./create-stock.component.css']
})
export class CreateStockComponent {

  public stock: Stock;
  public confirmed = false;
  public message = null;
  public exchanges = ['NYSE', 'NASDAQ', 'OTHER'];
  constructor(private stockService: StockService) {
    this.stock = new Stock('', '', 0, 0, 'NASDAQ');
  }

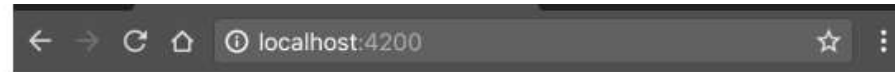
  setStockPrice(price) {
    this.stock.price = price;
    this.stock.previousPrice = price;
  }
}
```

```
createStock(stockForm) {  
  if (stockForm.valid) {  
    let created = this.stockService.createStock(this.stock); ③  
    if (created) { ④  
      this.message = 'Successfully created stock with stock code: '  
        + this.stock.code;  
      this.stock = new Stock('', '', 0, 0, 'NASDAQ');  
    } else {  
      this.message = 'Stock with stock code: ' + this.stock.code  
        + ' already exists';  
    }  
  } else {  
    console.error('Stock form is in an invalid state');  
  }  
}
```

THE EXAMPLE

- ❶ Add a message field to display success and error messages
- ❷ Inject `StockService` into the component
- ❸ Call `stockService.createStock` when the form is submitted
- ❹ Deal with success and error scenarios while creating the stock

THE EXAMPLE



Stock Market App

Test Stock Company (TSC) NASDAQ \$ 85 <button>Add to Favorite</button>	Second Stock Company (SSC) NSE \$ 10 <button>Add to Favorite</button>
Last Stock Company (LSC) NYSE \$ 876 <button>Add to Favorite</button>	My New Stock (MNS) NASDAQ \$ 244 <button>Add to Favorite</button>

Create Stock Form

Successfully created stock with stock code: MNS

Stock Name

Stock Name is Mandatory

Stock Code

Stock Code is Mandatory

0

NASDAQ

☒ I confirm that the information provided above is accurate!

Create

Stock Name is { "name": "", "code": "", "price": 0, "previousPrice": 0, "exchange": "NASDAQ", "favorite": false }

Data has been confirmed: true

STOCK MARKET

□ So, we accomplished:

- We created a StockListComponent to display a list of stocks.
- We created a StockService that acts as the layer behind all the components, providing APIs to fetch a list of stocks as well as to create new stocks. Currently, it is working off of mock data.
- We made updates to all the components to integrate with this service.

ASYNCHRONOUS OPERATIONS

- ❑ To fix data asynchrony using *observables*.
- ❑ Operators in Observable (RxJS Operators) are an extremely important part of working with Angular, especially when dealing with asynchronous data like HTTP requests, user input, or streams

ASYNCHRONOUS OPERATIONS

- ❑ What is an Observable?
 - It is a data stream that can emit values over time (asynchronous or synchronous).
- ❑ Often used to handle asynchronous events:
 - HTTP requests
 - User interactions (click, input)
 - WebSocket, timer...

ASYNCHRONOUS OPERATIONS

□ Main Categories of Operators

■ **Transformation:**

- `map()`: transform each emitted value.
- `switchMap()`: switch to a new Observable, cancel the previous one if a new value comes.
- `mergeMap()`: merge multiple Observables (often used with `async`).
- `concatMap()`: concatenate Observables sequentially.

ASYNCHRONOUS OPERATIONS

□ Main Categories of Operators

■ **Filtering:**

- `filter()`: filter values based on a condition.
- `take(n)`: take the first n values.
- `first()`, `last()`: take the first/last emitted value.
- `debounceTime(ms)`: ignore values emitted too quickly

ASYNCHRONOUS OPERATIONS

□ Main Categories of Operators

■ **Combination:**

- `combineLatest()`: combine multiple Observables, emit when any of them emits.
- `forkJoin()`: wait for all Observables to complete, then emit the results.
- `zip()`: combine values from multiple Observables in pairs

ASYNCHRONOUS OPERATIONS

□ Main Categories of Operators

■ **Error Handling:**

- `catchError()`: catch errors and return a fallback Observable.
- `retry(n)`: retry the source Observable up to n times on error

■ **Utility:**

- `tap()`: perform side effects (e.g., logging, debugging) without changing values.
- `delay(ms)`: delay emissions.
- `finalize()`: perform an action when the Observable completes.

ASYNCHRONOUS OPERATIONS

- ❑ Let's now take our example and move it step by step to using *observables* and make it ready for our future use cases.
- ❑ First, we will modify our StockService to start returning an *asynchronous observable* to get us ready for the future when we start integrating with servers.
- ❑ Then we will modify our components to subscribe to these observables and deal with success and error cases.

ASYNCHRONOUS OPERATIONS

- Let's change the *stock.service.ts* file as follows:

```
import { Injectable } from '@angular/core';

import { Observable } from 'rxjs/Observable';
import { _throw as ObservableThrow } from 'rxjs/observable/throw';
import { of as ObservableOf } from 'rxjs/observable/of';
import { Stock } from 'app/model/stock';

@Injectable()
export class StockService {

  private stocks: Stock[];
  constructor() {
    this.stocks = [
      new Stock('Test Stock Company', 'TSC', 85, 80, 'NASDAQ'),
      new Stock('Second Stock Company', 'SSC', 10, 20, 'NSE'),
      new Stock('Last Stock Company', 'LSC', 876, 765, 'NYSE')
    ];
  }
}
```

```

getStocks() : Observable<Stock[]> {
    return ObservableOf(this.stocks);
}

createStock(stock: Stock): Observable<any> {
    let foundStock = this.stocks.find(each => each.code === stock.code);
    if (foundStock) {
        return ObservableThrow({msg: 'Stock with code ' +
            stock.code + ' already exists'});
    }
    this.stocks.push(stock);
    return ObservableOf({msg: 'Stock with code ' + stock.code +
        ' successfully created'});
}

toggleFavorite(stock: Stock): Observable<Stock> {
    let foundStock = this.stocks.find(each => each.code === stock.code);
    foundStock.favorite = !foundStock.favorite;
    return ObservableOf(foundStock);
}
}

```

ASYNCHRONOUS OPERATIONS

- ❶ Importing Observable
- ❷ Import core methods from the Observable API, like `throw` and `of`
- ❸ Changing the return type of `getStocks` to an observable
- ❹ Returning an observable for mock data
- ❺ Throwing an exception to the observer

ASYNCHRONOUS OPERATIONS

- ❑ Now let's change the components to integrate with the new asynchronous APIs in the service.
- ❑ First, we will change the `StockListComponent`, to read the list of stocks from the observable instead of reading the array directly.
- ❑ We will change the *stock-list.component.ts* file as follows:

ASYNCHRONOUS OPERATIONS

- The *stock-list.component.ts* file

```
/** Imports skipped for brevity */
export class StockListComponent implements OnInit {

    public stocks: Stock[];
    constructor(private stockService: StockService) { }

    ngOnInit() {
        this.stockService.getStocks()
            .subscribe(stocks => {
                this.stocks = stocks;
            });
    }

    onToggleFavorite(stock: Stock) {
        this.stockService.toggleFavorite(stock);
    }
}
```

ASYNCHRONOUS OPERATIONS

- Let's also change the CreateStockComponent and see how the *create-stock.component* file changes:

```
/** Imports skipped for brevity */
export class CreateStockComponent {

  /** No changes, skipping for brevity */

  createStock(stockForm) {
    if (stockForm.valid) {
      this.stockService.createStock(this.stock)
        .subscribe((result: any) => { ❶
          this.message = result.msg;
          this.stock = new Stock('', '', 0, 0, 'NASDAQ');
        }, (err) => {
          this.message = err.msg;
        });
    } else {
      console.error('Stock form is in an invalid state');
    }
  }
}
```

❶ Subscribing to the observable

ASYNCHRONOUS OPERATIONS

- We'll change the *stock-list.component.ts*

```
/** Imports skipped for brevity */  
export class StockListComponent implements OnInit {  
  
    public stocks$: Observable<Stock[]>;  
    constructor(private stockService: StockService) { }  
  
    ngOnInit() {  
  
        this.stocks$ = this.stockService.getStocks();  
    }  
  
    onToggleFavorite(stock: Stock) {  
        this.stockService.toggleFavorite(stock);  
    }  
}
```

- ❶ Storing the observable as a member variable
- ❷ Calling and directly storing the observable

ASYNCHRONOUS OPERATIONS

- Given this, how do we display our array of stocks in the template? How do we handle this asynchronous behavior? Let's look at what we can do in the template to take care of this:

```
<app-stock-item *ngFor="let stock of stocks$ | async"  
    [stock]="stock"  
    (toggleFavorite)="onToggleFavorite($event)">  
</app-stock-item>
```

OBSERVABLE IN ANGULAR – SUMMARY

- ❑ **Observable** represents a stream of data that can emit values over time (next, error, complete).
- ❑ Angular's **HttpClient** always returns Observables, making it easy to handle async data.
- ❑ Using **of()** allows you to wrap static or mock data into an Observable, keeping consistency with real API calls
- ❑ Benefits:
 - Unified way to work with both mock and real data.
 - Can use **RxJS operators** (map, filter, catchError, etc.) to transform and control the stream.
 - Provides better flexibility, testability, and async handling

DEPENDENCY INJECTION

- ❑ Dependency Injection is an important part of the core set of Angular. Using Dependency Injection mechanism helps us to embed services into components or services together.
- ❑ Dependency injection allows us to create modular reusable pieces while allowing us to test components and modules easily

ANGULAR AND DEPENDENCY INJECTION

- ❑ To understand Angular's hierarchical dependency injection system by taking our example and modifying it to make the dependency injection system apparent.
- ❑ Before we get into the code. We will:
 - Add a new service called `MessageService`. This will be used to display messages at various points in the UI, and also as a mechanism to communicate across services.
 - Introduce and use the `MessageService` in both the `CreateStockComponent` as well as the `AppComponent`.

ANGULAR AND DEPENDENCY INJECTION

- First, use the Angular CLI to create the MessageService
 - **ng g service services/message --module=app**
 - This will create a skeleton MessageService in the *services* folder, and register it with the providers section of the AppModule.

ANGULAR AND DEPENDENCY INJECTION

- Let's now update the *message.service.ts* file as follows:

```
import { Injectable } from '@angular/core';

@Injectable()
export class MessageService {

    public message: string = null;

    constructor() { }
}
```

ANGULAR AND DEPENDENCY INJECTION

- ❑ Modify the AppComponent to use this service and display the current message in the UI. Modify the *app.component.ts* file as follows:

```
<h1>
  {{title}}
</h1>
<h3>App level: {{messageService.message}}</h3>
<app-stock-list></app-stock-list>
<app-create-stock></app-create-stock>
```

```
import { Component, OnInit } from '@angular/core';
import { MessageService } from 'app/services/message.service';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent implements OnInit {
  title = 'app works!';

  constructor(public messageService: MessageService) {}

  ngOnInit(): void {
    this.messageService.message = 'Hello Message Service!';
  }
}
```

ANGULAR AND DEPENDENCY INJECTION

- Next, we'll modify the CreateStockComponent to use the same service. Modify the *create-stock.component.ts* file as follows:

```
import { Component, OnInit } from '@angular/core';
import { Stock } from 'app/model/stock';
import { StockService } from 'app/services/stock.service';
import { MessageService } from 'app/services/message.service';

@Component({
  selector: 'app-create-stock',
  templateUrl: './create-stock.component.html',
  styleUrls: ['./create-stock.component.css'],
})
export class CreateStockComponent {

  public stock: Stock;
  public confirmed = false;
  public exchanges = ['NYSE', 'NASDAQ', 'OTHER'];
  constructor(private stockService: StockService,
               public messageService: MessageService) {
    this.stock = new Stock('', '', 0, 0, 'NASDAQ');
  }
```

- 1 Inject MessageService into the constructor
- 2 Use MessageService in both cases of creation

```
setStockPrice(price) {
  this.stock.price = price;
  this.stock.previousPrice = price;
}

createStock(stockForm) {
  if (stockForm.valid) {
    let created = this.stockService.createStock(this.stock);
    if (created) {
      this.messageService.message =
        'Successfully created stock with stock code: ' +
        this.stock.code;
      this.stock = new Stock('', '', 0, 0, 'NASDAQ');
    } else {
      this.messageService.message = 'Stock with stock code: ' +
        this.stock.code + ' already exists';
    }
  } else {
    console.error('Stock form is in an invalid state');
  }
}
```

ANGULAR AND DEPENDENCY INJECTION

- Finally, we will use the `MessageService` in the template of the component. Let's modify *create-stock/createstock.component.html* as follows:

```
<h2>Create Stock Form</h2>

<div>{{messageService.message}}</div>
<div class="form-group">
  <form (ngSubmit)="createStock(stockForm)" #stockForm="ngForm">
    <div class="stock-name">
      <input type="text"
        placeholder="Stock Name"
        required
        name="stockName"
        #stockName="ngModel"
        [(ngModel)]="stock.name">
    </div>

  <!-- Remaining code as before, omitting for brevity -->
```

ANGULAR AND DEPENDENCY INJECTION

- Now, let's modify the CreateStockComponent slightly as follows:

```
@Component({  
  selector: 'app-create-stock',  
  templateUrl: './create-stock.component.html',  
  styleUrls: ['./create-stock.component.css'],  
  providers: [MessageService] ❶  
})  
export class CreateStockComponent {
```

❶ Adding providers declaration for MessageService

```
  public stock: Stock;  
  public confirmed = false;  
  public exchanges = ['NYSE', 'NASDAQ', 'OTHER'];  
  constructor(private stockService: StockService,  
               public messageService: MessageService) {  
    this.stock = new Stock('', '', 0, 0, 'NASDAQ');  
    this.messageService.message = 'Component Level: Hello Message Service'; ❷  
  }
```

❷ Adding an initial value to the MessageService in the component

ANGULAR AND DEPENDENCY INJECTION

- ❑ Execute and note the following:
 - See the message in the AppComponent, which is the default we had set.
 - See the message in the CreateStockComponent, which will be the value we set in the component in the constructor "Component Level: Hello Message Service".
 - Now fill up the form and create a stock
 - Note that the message is updated only in the CreateStockComponent, but the AppComponent message does not change

Stock Market App

App level: Hello Message Service!

Test Stock Company (TSC) NASDAQ \$ 85 Add to Favorite	Second Stock Company (SSC) NSE \$ 10 Add to Favorite
Last Stock Company (LSC) NYSE \$ 876 Add to Favorite	Test (TSS) NASDAQ \$ 44 Add to Favorite

Create Stock Form

Successfully created stock with stock code: TSS

Stock Name is Mandatory

Stock Code is Mandatory

☒ I confirm that the information provided above is accurate!

Stock Name is { "name": "", "code": "", "price": 0, "previousPrice": 0, "exchange": "NASDAQ", "favorite": false }

Data has been confirmed: true